

Beagle: Auto-tuning Performance Diagnosis for Live Video Streaming

Chandan Bothra	Leonardo Teixeira	Zahaib Akhtar	Sanjay G Rao	Bruno Ribeiro
<i>Purdue University</i>	<i>Purdue University</i>	<i>Amazon and NCSU</i>	<i>Purdue University</i>	<i>Purdue University</i>
West Lafayette, USA	West Lafayette, USA	San Francisco, USA	West Lafayette, USA	West Lafayette, USA

Abstract—Delivering high-quality live video to millions of users is challenging, and performance can be impacted by a wide range of components in the ecosystem (e.g., CDNs, ISPs, protocols) as well as how these components interact with each other. In this paper, our primary contribution is Beagle, a novel system for root-causing performance anomalies in the video streaming context. As part of Beagle, we developed custom interpretable ML models based on mechanisms used by domain experts (e.g., filters based on rules and a graph relating those rules). Leveraging pre-training and few-shot learning, Beagle supports automatic learning and adaptation of rule thresholds and graph relationships from limited ground truth information. Evaluations with real-world dataset of live streaming sessions from a popular streaming service show (i) Beagle significantly outperforms alternatives including Gradient Boosted Decision Trees, and a Deep Neural Network approach; and (ii) Beagle is effective in auto-tuning production expert designed system for root cause identification. Even with less than ten root causes to learn from, Beagle reduces the Mistake Rate made by expert configured rules by nearly an order of magnitude.

Index Terms—ML for Systems, Root cause analysis

I. INTRODUCTION

Video providers stream video to tens of millions of users concurrently (e.g., concerts, large sports events). Ensuring a high quality of experience (low rebuffering, high bit rates) is critical [1]–[3], with users that span a variety of Internet Service Providers (ISPs), Content Delivery Networks (CDNs), different geographies, and using a wide range of devices, players, and protocols. An issue that impacts any single component in the ecosystem (e.g., a particular player or protocol, or ISP in a given city) can impact performance for many users. Poor performance experienced by even a small subset of users is unacceptable and requires prompt action when feasible (e.g., rerouting users to a different CDN) [4].

Given the stakes, video providers invest significant resources on real-time performance diagnosis, which is challenging given the need to process large volumes of data related to user performance that stream in continually. Further, data is often aggregated in a coarse manner (for scalability and to ensure only statistically significant issues are tracked), which limits visibility into the data. Finally, anomalies in seemingly unrelated user groups may correspond to the same root cause. For instance, a performance problem associated with a video streaming protocol may also seem to indicate a problem with a CDN if users in the CDN typically use that particular protocol.

It is common for video providers to have a variety of systems to help diagnose performance anomalies [5]–[7]. These include monitoring performance of different groups of users, filtering anomalies based on rules that indicate which anomalies are critical, and using graph-based filters that indicate certain anomalies must take precedence over others [8]–[19]. Maintaining these systems requires careful tuning (e.g., thresholds, precedence relationships) guided by extensive domain knowledge. Further, the tuning must occur continuously over time to reflect past experience and the changes need to be carefully vetted, involving significant “human in the loop”, and long production cycles. Finally, feedback is often limited, since many types of performance problems occur relatively rarely. Ground truth information requires manual analysis and feedback from operational teams and is only available for a small subset of predictions.

Contributions. Beagle addresses several challenges. First, Beagle learns precedence relationships from data to allow correlation across potential root causes. Beagle models this challenging combinatorial graph learning problem as a continuous optimization problem [20]. The graph that Beagle learns captures more general relationships (e.g., relating device families, and players) although the data may be more specific (e.g., relating a specific device family and a specific player). Second, Beagle models diagnosis rules that often involve multiple logical operations [10], [21]. (e.g, an anomaly event triggers an alarm only if multiple other anomaly events that have a higher precedence do not occur). Beagle does so by continuous approximations of the discrete logical operators to make them suitable for gradient-based optimization. Third, to deal with limited feedback or ground truth, Beagle uses a combination of self-supervised pre-training [22] and few-shot learning [23], [24] with carefully tuned loss functions that match available ground truth labels. Our self-supervised pre-training ensures that Beagle matches an existing policy when feedback is unavailable.

Results. We evaluate Beagle using two large-scale sampled datasets of live streaming sessions (corresponding to widely watched sports) collected from **ANON**, a popular streaming service, collected over multiple days across months, and have over a million concurrent sessions. Our results show that:

- Beagle is effective in auto-tuning and outperforms alternatives such as gradient boosted decision trees (GBDT) and a deep neural network approach (Set Transformer). When primed with

TABLE I: Dimensions associated with an user session, and order of magnitude of unique values for each dimension.

Dimension	Unique	Dimension	Unique
Country	≈ 10	Origin	≈ 10
CDN	≈ 10	Player	≈ 10
ASN	$\approx 10^4$	Protocol	< 10
City	$\approx 10^4$	Device Family	$\approx 10^2$

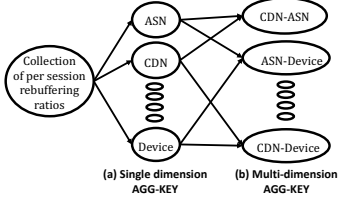


Fig. 1: Single & multi-dimension aggregation.

a wide set of prior models with F1 scores as low as 0.19, Beagle auto-tunes the models to achieve F1 scores of 0.96 or higher. In contrast, GBDT and Set Transformer only achieve an F1 score of 0.35 and 0.52 respectively.

- An “in the wild” case study shows Beagle reduces errors in performance diagnosis by nearly an order of magnitude overall even when primed with a carefully tuned expert model used in production, and when only provided with a small amount of historical feedback.

II. BACKGROUND AND MOTIVATION

Large scale video publishers concurrently stream millions of video sessions daily. Ensuring high user experience is critical for all users, and poor performance even if in the tail is unacceptable. Performance is monitored by continually collecting video delivery metrics (e.g. rebuffering ratio, average bitrate etc.) of user sessions at regular intervals.

Each session is associated with contextual information such as the Autonomous System Number (ASN), CDN, streaming protocol and device etc. which we refer to as dimensions. Table I summarizes the set of dimensions, and provides an order of magnitude for the number of unique values associated with each dimension for **ANON**, a popular streaming service (anonymized). For instance, user sessions associate with over 10K unique ASNs and cities, and over hundred device families.

Performance problems across users may be correlated – e.g., they may pertain to users associated with a particular device family, or CDN. To detect anomalies at scale, performance statistics may be aggregated over a dimension, or a combination of dimensions, which we refer to as an *aggregation key* and abbreviate as **AGG-KEY**. Figure 1 (a) and (b) respectively show examples of aggregating over a single dimension **AGG-KEY**, and an **AGG-KEY** that involves a combination of dimensions.

A. Challenges in root causing anomalies

Any performance anomaly that impacts a significant group of users must be detected and resolved live (within a few minutes). Thus it is important that relevant operational teams are identified and engaged expeditiously. For example, a diagnosis

that indicates an issue with a particular CDN, and a diagnosis indicating an issue with a particular device family typically requires actions by two completely different teams. Given resolutions steps are time and labor intensive, accurate diagnosis is critical, however, this diagnosis is challenging on several fronts as described below.

Aggregation vs Detail Tradeoff. The high cardinality of dimensions shown in Table I result in a combinatorial explosion of possible user groups. Detecting anomalies on all these user groups is prohibitively expensive, thus system architects need to strike a tradeoff where **AGG-KEYs** are carefully selected and restricted to a small number of dimensions. This in turn provides only coarse views of data. For instance, using CDN as an **AGG-KEY** allows identification of issues pertaining to the an entire CDN, but not issues that only impact a subset of CDN users (say CDN users in a given ASN).

Dependence between Dimensions. A single performance issue can cause several different user groups to appear anomalous. For example, consider a problem that impacts a particular combination of player P and protocol Q. Further, player P is usually used on a set of device families (DF_1 and DF_2). The protocol Q itself is supported only on some combination of CDNs (CDN_A , CDN_B) and origins (O_1 , O_2 and O_3). The performance problem manifests in several user groups spanning multiple **AGG-KEYs** as shown in the figure. Comprehensively capturing such dependencies, even with extensive domain expertise is non-trivial and results in significant complications for system architects.

Needle in a Haystack Problem. The high cardinality of individual dimensions (e.g. 10^4 ASNs) result in a large number of potential anomalies which must be analyzed in real time. The problem is further exacerbated since anomalies may be tracked across different metrics (e.g., rebuffering rate, average bit rate, fatal errors etc.). Each additional metric increases the workload size by a multiplicative factor. As such, it is common to have over $\approx 10K$ flagged anomalies at any time step. Finding the true root cause is then akin to finding a needle in a haystack where large amounts of spurious root causes must be distilled out for a conclusion.

B. Why existing approaches fall short?

Prior work has looked at isolating performance problems associated with video quality [5]. A group of sessions (e.g., corresponding to a CDN or a CDN-ASN combination) is identified as anomalous if the fraction of sessions that see poor performance in that group exceeds a threshold. Further, user groups exhibit parent-child relationships which are leveraged to isolate the root cause. A group is identified as a root cause if all of its children are anomalous but its parents are not.

While [5] is a useful starting point, it suffers from several issues. First, threshold to identify whether a group of sessions is anomalous is set statically: high values may result in false negatives, and low values in false positives. Second, not all children of a true root cause may be anomalous owing to skewed distribution of sessions. For example, if CDN1 is

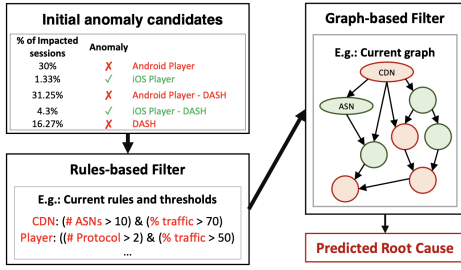


Fig. 2: Filtering anomaly candidates through rule-based and graph-based filters.

the true root cause, is possible that CDN1-ASN2 may not be flagged as anomalous if fraction of poor sessions in that group is under the threshold. This in turn impacts the root cause isolation algorithm. Finally, determining parent-child relationships using simple rules described above is insufficient due to non-trivial correlations between dimensions. For example, a performance issue associated with player P and protocol Q manifests in several other user groups owing to player P being closely associated with a set of device families, and protocol Q being closely associated with some CDNs and origins.

Our discussion with practitioners indicate that while approaches similar to [5] are a good starting point, extensive domain knowledge and continual manual tuning is needed to suppress spurious alerts in practice. Specifically, production troubleshooting systems require configuration of *rules* and *precedence relations* to filter anomalies. Rules indicate which anomalies must be prioritized. For example, a rule might indicate that an anomaly related to a CDN only merit further attention if sufficient sessions in the CDN are impacted, and they belong to a sufficiently diverse set of ASNs. Precedence relations capture correlations that practitioners observe in practice. For instance, if sessions corresponding to a player, as well as a closely related Device Family are anomalous, the precedence relation captures whether one of the anomalies must be suppressed. The configuration process requires domain expertise and extensive tuning and periodic updates to determine correct rule thresholds and precedence relationships.

C. Beagle’s approach

Beagle is a novel approach for auto-tuning systems for anomaly localization with multi-dimensional data, especially video streaming. Figure 2 shows how Beagle abstracts anomaly localization systems such as those discussed above. Given an initial candidate set of anomalies, Beagle models (i) rule-based filters which indicate which anomalies need further consideration; and (ii) graph-based filters which suppress anomalies based on correlations across user groups. Graph-based filtering involves modeling a dependency graph (a directed acyclic graph or DAG) across **AGG-KEYs**, and deciding which anomalies to suppress based on parent-child relationships.

The key novelty of Beagle is that it automatically tunes rule thresholds and learns parent-child relationships based on feedback from past diagnosis episodes. Such feedback is obtained through *retrospectives* conducted by relevant operational teams, where human operators evaluate whether the

predicted root cause was accurate. As such, these retrospectives are typically available for a small subset of the anomaly predictions. To achieve these goals, we have developed a custom ML model as part of Beagle. The model is designed to be **interpretable**, and **aligned** with the filtering mechanisms used by domain experts.

III. BEAGLE DESIGN OVERVIEW

Beagle consists of the following components:

Learning thresholds in rule-based filters. Beagle models rule based filters, which can be viewed as multiple propositions combined with logical operations (AND, OR). Each proposition consists of a feature present in the data and a threshold. For instance, to flag **CDN_C** as anomalous, a rule-based filter may require that (i) at least X% (where X is configurable) of the sessions served by **CDN_C** must be impaired and (ii) that these impaired session must be observed across at least Y% ASNs (where Y is configurable) served by **CDN_C**. To support learning such discrete filters via gradient descent, Beagle models logical operations through continuous relaxation, for which gradients can be computed.

Learning graph-based filters. Beagle learns parent and child relationships between **AGG-KEYs** which indicate an anomaly with the parent or child may be filtered if certain conditions are met. It supports rules of two kinds (i) *ChildFiltersParent* rules, which indicate that if child **AGG-KEY** and parent **AGG-KEY** are both initially anomalous, then only the child is retained anomalous; and (ii) *ParentFiltersChild* rules, which indicate that a child **AGG-KEY** is only retained as anomalous if the parent **AGG-KEY** is also anomalous. To tackle this, we first model filters used by domain experts (which rely on conditional statements and loops for a known graph) as logical operations in Beagle that work even when graph relationships are unknown and must be learnt. Further, rather than express the presence or absence of an edge as a discrete logical operator, Beagle models this as a continuous approximation. Finally, Beagle uses logical operations that consider all possible edges to model the necessary conditions.

Learn general rules from specific patterns. Although only a specific anomaly may have been seen in the data, it is important to extrapolate the pattern to other similar anomalies that may not be present in the data. In the rule-based filtering, we don’t learn a rule that is specific to a single datapoint (**DF_A-Origin_D** and **Player_A**), but rather, attempt to learn a rule that works well with any data point (DF-Origin and Player). This way, we not only avoid over fitting to specific examples in our data, but also can generalize to unseen values (for example, the rule that we learn can be applied in the future for new DFs, Origins and Players or even new combinations of DF-Origin and Player that were previously not observed).

We also follow similar approach of learning general rules for our graph-based filtering. For instance, consider an example showing that when **Player_A** and **DF_A-Origin_D** are both anomalous, and when a significant majority of **Player_A** sessions belong to **DF_A-Origin_D**, the more precise anomaly (**Player_A**)

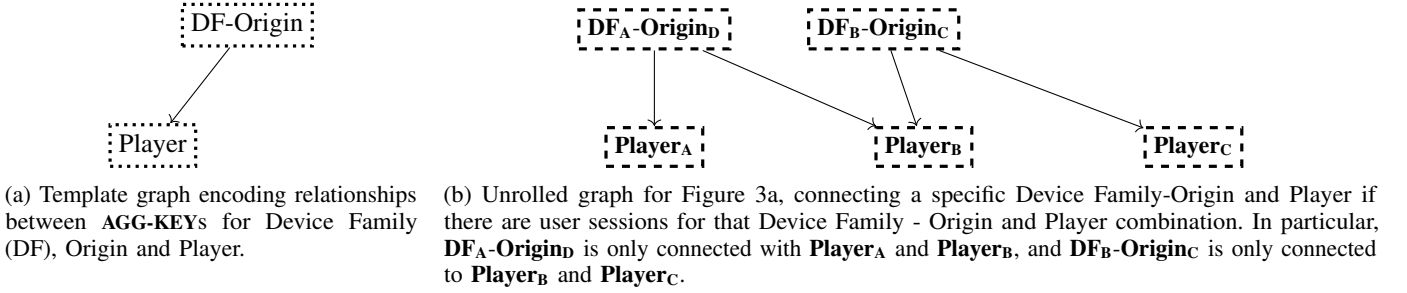


Fig. 3: Illustration of unrolled graph (b) built from the template and observed data (a).

is retained, and the more onerous **DF_A-Origin_D** is filtered. In such cases, Beagle learns a graph relationship between Players and DF-Origins more generally rather than merely relate the specific **Player_A** and specific **DF_A-Origin_D**. This ensures that when a similar pattern is encountered for a different Player and DF-Origin, Beagle can identify the root cause in a similar fashion. To achieve this, Beagle learns relationships on a *template graph* based on **AGG-KEYs** (relating **AGG-KEYs** Player and DF-Origin), rather than on relationships on specific values of **AGG-KEYs**. When making a decision for a given minute of data, this template graph is unrolled to a larger version where the nodes represent the actual (specific) values (e.g. **Player_A** and **DF_A-Origin_D**) observed in the data. Figure 3 illustrates how the template graph is unrolled. The learning algorithm operates on specific values (unrolled graph), but learns generic rules using shared parameters (template graph).

Learning cascaded sequence of filters. In practice, domain experts may combine rule-based and graph-based filters in sequence. Beagle supports learning across a sequence of filters, where it simultaneously learns thresholds in rule-based filters, and relationships between **AGG-KEYs** in graph-based filters. A key issue is that any feedback is only available for the final result (after the combination of filters), which reduces the signal available to the models earlier in the process. Beagle handles this by (i) supporting different learning rates for different filters; (ii) using a loss function that explicitly considers the earlier stages, and inferring ground truth for the earlier stage in certain cases (e.g., a feedback regarding a particular root cause implies that it should have remained an anomaly in every preceding filter stage). More details are discussed in §IV-E.

Learning from limited feedback. A key challenge for Beagle is that only limited feedback (in the form of retrospectives) is available. Further, the root causes encountered in practice exhibit significant imbalance, and there is also much imbalance. To handle this, we carefully tailor Beagle’s loss function based on whether feedback is available for a particular prediction, and based on whether the ground truth is a true root cause or not. The loss function also aims to balance across root causes, and across positive and negative examples. Given that no feedback is available for a large number of samples, Beagle can potentially arbitrarily tune thresholds and graph relationships given many possible configurations may fit the data points for which feedback is available. To handle this, Beagle’s loss

function models that the existing parameters (e.g., provided by a domain expert) usually give the correct result for data points with no feedback, and applies a lower weight (a tunable hyper-parameter) for these data points relative to those where feedback is available.

IV. BEAGLE DESIGN DETAILS

We now present details of Beagle’s design. The model input is a sequence of minutes $\mathbf{X} = (\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(T)})$, with each minute $t \in \{1, \dots, T\}$ composed of a sequence $\mathbf{X}(t)$ of n_t points $(x_{t,1}, \dots, x_{t,n_t})$, where each data point corresponds to a potential anomaly. For example, $x_{t,i}$ might denote the set of features for **CDN_A-ASN_B** (of type CDN-ASN) along with an indication of whether there was an initial anomaly marker for that feature. Associated with each data point $x_{t,i}$ is a label $y_{t,i} \in \{0, 1\}$, indicating if that data point corresponds to a true root-caused anomaly or not (if feedback for that data point is available). In what follows, we may drop the indices t and/or i for simplicity of notation when referring to functions or equations that apply to a given minute or a given data point.

A. Modeling rule-based filters

Each **AGG-KEY** has its own rule, taking as input multiple numerical features and a binary feature (indicating whether the time-series analysis flagged that data point as anomalous). For each numerical feature (of each rule), we have a threshold parameter, against which the feature value is compared, using a step function. Example of possible rules is shown in Figure 2. The final output of the rule is the result of a propositional logic statement (ANDs and ORs of the comparison of each feature).

More formally, let $\mathcal{D}$ be the set of all **AGG-KEYs** (e.g. $\{\text{CDN}, \text{CDN-Protocol}, \dots\}$). Further, assume a function $d(x_i)$ that associates an input data point x_i with its corresponding **AGG-KEY**. With each **AGG-KEY** k we associate a vector of thresholds $\theta_k = (\theta_{k,1}, \dots, \theta_{k,p})$ where p denotes the number of numerical features. These thresholds are used in the rule for that **AGG-KEY**, which we denote by h_k , that is a logical proposition, composed by ANDs and ORs of clauses $h_{k,j}$ (with $j \in \{1, \dots, p\}$), one for each feature. These clauses compare the feature value with a threshold (e.g. $h_{k,j}(x_i, \theta_{k,j}) = \mathbb{I}[x_{i,j} > \theta_{k,j}]$). The result of the rule applied to a given data point x_i will be whether the data point must

be suppressed (0) or not (1). For brevity, we denote this result by $h(x_i, \theta)$.

Continuation of Logical Operations and other approximations. The filters of Beagle (both the rules and graph based filter) can be described mathematically as composition of functions that are discrete in nature (e.g. logic operations usually operating on boolean values) or that have discontinuities (e.g. the step function used for thresholding). However, to be able to learn the parameters of these functions, we need to approximate these discontinuous and discrete functions with a continuous counterpart, which we can use with standard gradient based ML methods.

B. Learning rules on a templated graph

In the graph-based filter, we model two types of filters: *ChildFiltersParent* and *ParentFiltersChild*. Both of these rules relate two data points of distinct **AGG-KEY**, for example, the data point for **DF-Origins_D** with the data point for **Player_A**. In order to learn relationship rules that generalize across multiple similar data points, these rules are modeled as generic rules on a *template* graph, whose nodes denote the generic **AGG-KEYs** (such as DF-Origin and Player), and their relationships. Please refer to Figure 3a for an example.

Formally, let $\mathbf{A} \in [0, 1]^{|\mathcal{D}| \times |\mathcal{D}|}$ be the template adjacency matrix, from which we derive the unrolled adjacency matrix $\hat{\mathbf{A}}^{(t)} \in [0, 1]^{n_t \times n_t}$ for each minute, where $\hat{A}_{i,j}^{(t)} = A_{d(x_{t,i}), d(x_{t,j})}$ if $x_{t,i}$ and $x_{t,j}$ are related or 0 otherwise. The edge value goes from 0 (no edge) to 1 (edge is present). We also indicate by $B_{i,j}$ if the edge from **AGG-KEY** i to j denotes a *ParentFiltersChild* rule ($B_{i,j} = 1$), in addition to the *ChildFiltersParent* rule (applicable to all edges). Note that edge parameter $A_{i,j}$ is shared for all edges of between nodes of **AGG-KEY** i and j .

C. Modeling graph-based filters

For each node $x_{t,i}$ in the unrolled graph for a given minute t , we compute the following functions that indicate whether node i should be suppressed by any of its children or by any of its parents. The child suppression, indicated by $c_i(\mathbf{X}^{(t)})$ happens whenever any child is anomalous. The parent suppression, indicated by $p_i(\mathbf{X}^{(t)})$ happens when (i) any parent is not anomalous; and (ii) the edge applies the *ParentFiltersChildren* rule. The functions are as follows:

$$c_i(\mathbf{X}^{(t)}) = \bigvee_{j=1}^{n_t} [\hat{A}_{i,j} \wedge h(x_j, \theta)]$$

$$p_i(\mathbf{X}^{(t)}) = \bigvee_{j=1}^{n_t} [\hat{A}_{j,i} \wedge B_{j,i} \wedge \neg h(x_j, \theta)].$$

For each data point, the final prediction is computed as:

$$f_i(\mathbf{X}^{(t)}) = h(x_i, \theta) \wedge \neg p_i(\mathbf{X}^{(t)}) \wedge \neg c_i(\mathbf{X}^{(t)}) \quad (1)$$

In the graph-based filter, while the filtering type $\mathbf{B}$ (parent/child suppression) of each edge is fixed, the graph structure $\mathbf{A}$ is learned and it presents two main challenges. First, we need to turn it from a combinatorial optimization to a gradient-based optimization. Second, since we need to avoid two nodes filtering

each other, we need to learn a Directed Acyclic Graph (DAG). We address these challenges by employing the same relaxation strategies for boolean logic used for the rules-based filter. Next, we propose a graph learning method inspired by Zheng et al. [20], which treats the adjacency matrix as continuous values in the range $[0, 1]$ and introduces a ‘‘DAGness’’ constraint, which penalizes graphs with cycles with strong edges (edges close to 1). This constrained optimization is solved via the augmented Lagrangian method [25], which involves solving a sequence of unconstrained optimization as we discuss in §IV-E.

D. Learning sequence of filters

Beagle applies the graph filter in sequence after the rule-based filter (as done in real world). The learning of both filters is done *end-to-end*, i.e. since we don’t have ground-truth for the rules-based filter alone, its parameters are learned through the backpropagation of mistakes made at the final prediction (after graph filter). As a consequence, each mistake may give feedback for more than one rule (as the mistake is propagated to both parent and children of a node).

One strategy we use to facilitate this end-to-end learning exploits the fact that, for positive (anomalous) data points, the output of the rules-based filter must also be positive (since the graph filter can only suppress it further), as a direct consequence of Equation 1. For these (true positive) data points we can add a penalty term in the loss for when the rules-based filter deviates from a positive prediction.

A second strategy is to employ a combination of different learning rates for the parameters of both filters (rules- and graph-based); and treating those two sets of parameters as two separate vectors which we then re-scale independently, so that the gradients for one part of the model are not washed out just as consequence of scaling all parameters jointly.

E. Tuning loss for limited feedback

Even though the data collected for root cause diagnosis is large, the number of actual positive samples (anomalies) is typically low. Moreover, annotating every single prediction can be an expensive and time-consuming effort, so we are restricted on learning from the manually annotated feedback from *retrospectives*, where only the anomalies predicted by Beagle are reviewed by human experts. Additionally, the number of samples (and mistakes) can vary significantly (by two orders of magnitude) across different root causes, e.g. due to recurrence of the same issue or defect in a particular component of the video delivery pipeline.

Loss weights. We handle skewed distribution of positive and negative samples and the number of data points across root causes by balancing loss across root causes and then within each root cause, this weight is balanced for the positive and negative samples. We denote by $w_{k,j}$ the weights for **AGG-KEY** $k \in \mathcal{D}$ and for class $j \in \{0, 1\}$.

Regularization. We add a regularization term to our loss function, to encourage the rules-based filter to match the original policy for those data points without ground-truth. This

prevents our learning procedure to arbitrarily change the model and overfit the limited set of labeled data points.

Loss function. In what follows, we describe our loss function in more detail. We denote the set of data points for which feedback is available as $X^{(Retro)}$, the set of data points for which the ground-truth label is positive by $X^{(=1)}$ and the set of data points with negative ground-truth label or for which the feedback is not available as $X^{\neq 1}$. As discussed in §IV-C, to enforce the DAGness, our optimization becomes the following max-min problem:

$$\max_{\alpha} \min_{\mathbf{A}, \boldsymbol{\theta}} \mathcal{L}_{RC}(\mathbf{A}, \boldsymbol{\theta}) + \mathcal{L}_R(\boldsymbol{\theta}) + \mathcal{L}_{REG}(\boldsymbol{\theta}) + \mathcal{L}_{DAG}(\mathbf{A}, \alpha),$$

where α is the dual parameter¹. The term $\mathcal{L}_{RC}$ refers to the penalty of the mistakes made wrt to the available ground-truth label, $\mathcal{L}_R$ is the penalty for when the rules-based filter wrongly suppresses the data points with positive ground-truth label, $\mathcal{L}_{REG}$ is the regularization term and $\mathcal{L}_{DAG}$ arises from employing the augmented Lagrangian method to solve the constrained optimization.

V. EVALUATION

We evaluate Beagle with a view to answering the following:

- How effective is Beagle’s auto-tuning approach relative to alternative approaches? To this end, we compare Beagle with a classical ML approach (GBDTs), as well as a state-of-the-art Deep Neural Network, *Set Transformer* (§V-A).
- How effective is Beagle in auto-tuning a real-world diagnosis system? We present a case study demonstrating Beagle’s ability to auto-tune a production root causing system (§V-B).

Datasets. We conducted evaluations on two large scale datasets (**Dataset 1** and **Dataset 2**), both obtained from live streams of two popular sports. Both datasets collected data from multiple games conducted over a season with each game lasting for multiple hours. Across the datasets, the number of concurrent sessions was greater than 1 million, spanning multiple dimensions (ASNs, CDNs, device etc.) (§II) with several hundreds of thousands of groups of session based on these dimensions. Each minute of data is summarized across several **AGG-KEYs** (e.g. CDN-ASN). For each specific value (e.g., **CDN_A-ASN_A**, **CDN_A-ASN_B** etc.) of each **AGG-KEY** (which we may refer to as **AGG-VAL**), the relevant features (e.g., fraction of sessions impacted by anomaly) for that minute are included along with a boolean flag to indicate whether a performance anomaly was initially suspected for sessions corresponding to that **AGG-VAL**.

No ground truth of root causes was available for **Dataset 2**. For **Dataset 1**, limited ground truth was available (only 1.13% of all data points) and was obtained as follows: (i) an expert designed production root causing system predicted probable root cause of anomalies; (ii) for a subset of cases when a positive prediction was made, human operators performed

manual *retrospective* analysis of these root causes which helped generate ground truth labels.

Schemes compared. We compare Beagle with the following baselines, one of which models operational practice, and two of which are well known ML models which serve as baselines:

- **Expert:** This scheme captures a manually tuned prior system for diagnosis, which also serves as an initialization of Beagle prior to its auto-tuning. Our focus here is on evaluating whether Beagle can provide benefits over a relatively well tuned Expert in the real world.

- **GBDT:** We use Gradient Boosted Decision Trees [26], or GBDTs, a classical model for classification tasks where the input data is structured (i.e., organized in predefined format, such as tables). We train a GBDT per **AGG-KEY** that takes numerical features relevant to that **AGG-KEY**. Note the approach cannot make collective decisions across root-causes (e.g. **CDN_C** and **CDN_C-ASN_A** may be independently flagged as root-causes and the model cannot make joint decisions across them).

- **Set Transformer:** We use a state-of-the-art set-based Deep Neural Network, *Set Transformer* [27], which is capable of learning complex associations in unstructured data streams. A Set Transformer can operate on a set of data points and make decisions using all the (variable-sized) set of inputs available at any given minute. However, these models are much less interpretable and usually require a lot of data to train. Specifically, for each minute, we provide a set where each element corresponds to a one-hot encoding of a corresponding **AGG-VAL** (e.g. **CDN_A**, **ASN_A**, etc.) and its relevant features. The output of the set transformer is fed to a classifier layer (a multi-layer perceptron) whose output is a bit vector, where each bit corresponds to a prediction of whether a particular **AGG-VAL** is a root cause.

Training setup. We optimize Beagle using gradient descent algorithm with Adam optimizer [28] and Cosine Annealing learning rate scheduler [29]. We run grid-search hyper parameter tuning for different learning rates and regularization to match the existing policy for data without ground truth feedback). We build GBDT using the XGBoost [30], and run hyper parameter search across learning rates, tree depth and sub sample ratio. We use standard implementation of transformer in PyTorch library to build the Set Transformer with hyper parameter search across different learning rates, embedding dimension and number of layers in the transformer. We run experiments on NVIDIA Tesla V100 GPUs in AWS Sagemaker.

A. Beagle vs Alternative approaches

Methodology. As discussed earlier, our datasets have no or highly limited ground truth information. To facilitate comparisons, we generate simulated ground truth for this dataset, by building an *oracle* model, i.e. a version of our model where we select the filters (rules’ thresholds and graph structure), which are kept hidden, but whose output we make available as the true labels for each data point. Our evaluations focus on the **Dataset 2** dataset. We use the first 60% of the

¹from the Lagrangian method

TABLE II: Test suite to validate Beagle’s auto-tuning abilities.

Test case	Description
R-1, R-2, R-3	Modify thresholds of rule-based filters
NoEdges	No edges in the graph
IsolatedNode	Disconnect one of the nodes in the graph
Del2Edges	Delete two existing edges from the graph
AddDelEdge	Remove an existing edge and add a spurious edge in the graph

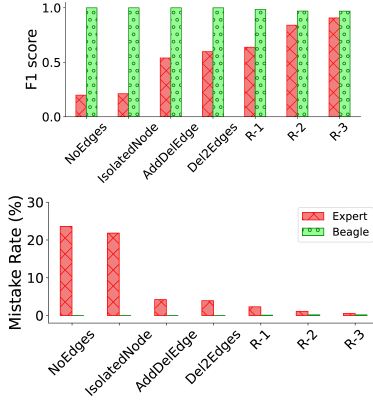


Fig. 4: F1 score and Mistake Rate for various test cases validating Beagle’s auto-tuning abilities.

minutes for training, the next 20% for validation and the final 20% for testing for all the schemes.

Evaluating Beagle’s auto-tuning ability. We create an extensive set of Expert models by perturbing the *oracle* model. These perturbed models serve as the initial state of Beagle, and are used to evaluate Beagle’s ability to auto-tune filters and graph relationships. The perturbations for rule-based filters involved changing the thresholds of one or more features for one or more **AGG-KEY**, while perturbations for graph-based filters involved changing the structure of the graph used in the *oracle* by either adding or removing edges (Table II). The *oracle* model contained nine **AGG-KEY**s in the template graph (over fifteen hundred nodes in the unrolled graph). In these experiments, Beagle needed to auto-tune several tens of thresholds, and the weights for several tens of edge relationships.

Figure 4 reports the F1 score, and Mistake Rate with the different Expert models, and Beagle. The Mistake Rate is defined as the sum of errors (False Positives and False Negatives) divided by the total number of samples. We observe that the Expert models show a wide range of performance based on the degree of perturbation of the oracle. However, in all cases, Beagle even when primed with relatively weak Expert models achieves an F1 scores of nearly 1 in all cases by auto-tuning the rule-based and graph-based filters. Further, it achieves a Mistake Rate of under 0.01% for most test cases, and reduction of orders of magnitude in some cases.

Comparison results. We next compare Beagle with GBDT and Set Transformer. We focus our results on a setting where Beagle was primed with the weakest Expert (NoEdges, where the Expert does not model any graph relationship).

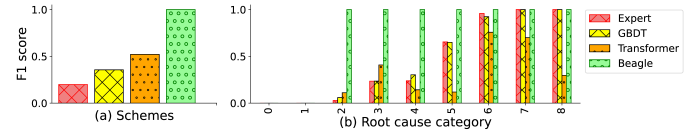


Fig. 5: Comparing overall F1 score (left) for different schemes, and break down by root cause category (right). The two left most root cause categories do not have positive ground truth labels and an F1 score does not provide meaningful information.

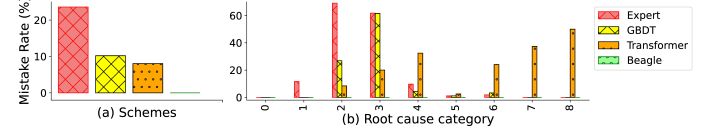


Fig. 6: Comparing Mistake Rate for different schemes (left), and breakdown by root cause category (right).

Figure 5 and Figure 6 show the overall F1 score and Mistake Rate for the different schemes along with a breakdown for each category of root cause labels. We make several points. First, when overall performance is considered, Beagle outperforms GBDT and Set Transformer achieving a significantly higher F1 score and lower Mistake Rate. Beagle is able to do so despite being primed with an Expert that performs quite poorly. Second, GBDT achieves the lowest F1 and highest Mistake Rate. The breakdown by category shows that GBDT performs particularly poorly for two of the root cause categories (2 and 3). Further analysis showed that achieving good performance for these categories requires capturing graph relationships which GBDT does not. Finally, Set Transformer does not perform significantly better despite capturing correlations, an aspect that we investigate further in depth.

Why does the Set Transformer fall short? A potential hypothesis for why the Set Transformer did not perform as well is changes in distribution between data used for training, and the data encountered by the live system (when performing root-cause diagnosis) [31]. Under such shifts, mistakes that were unseen or rare during training may become more common.

To understand if distribution shifts may have played a role, we first examine whether the Set Transformer is inherently capable in the absence of shifts. To do this, we mix past and future data into a single dataset and partition it randomly into a new training and test data (which are likely to have the same distribution owing to mixing).

Figure 7a compares the performance of the Set Transformer when training over the mixed dataset and when training using the past (chronological) data alone. The figure shows that the F1 score for the root causes in the mixed set is significantly higher (with a median improvement of 68%), compared to the original (chronological) set.

To gain further evidence of a distribution shift, we analyze the clustering of embeddings produced by the Set Transformer when trained chronologically for the training and testing sets [32]. The embeddings map each minute’s data to a single point in a high-dimensional space. We cluster the embeddings across minutes using the *k*-means algorithm. We then map the

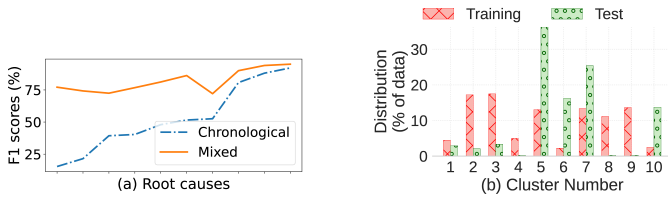


Fig. 7: (a) Set Transformer achieves a better F1 score for all labels when training on a mixed dataset (past + future) versus chronological (past only), due to distribution shift. (b) We observe distribution shift between past and future data by analyzing embedding clusters from a Set Transformer trained only on past data.

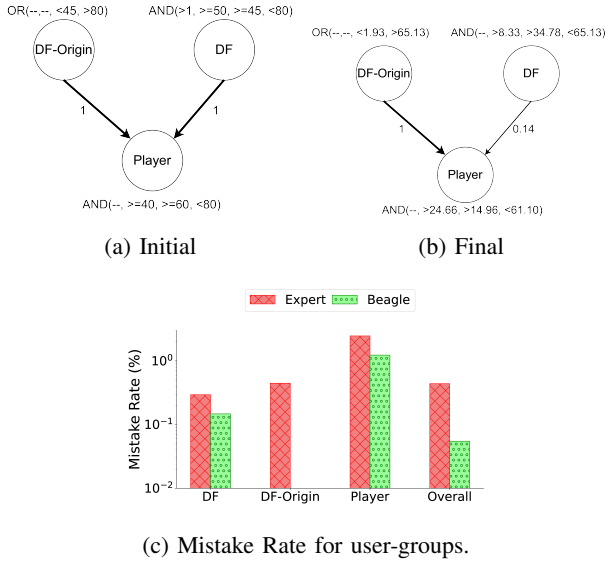


Fig. 8: Comparing the initial and final rules and graph based filters, along with the Mistake Rate.

original test data into the same k clusters and compare the histogram of the training (past) and test (future) data. Figure 7b presents the comparison. There are two bars for each cluster, representing the percentage of the dataset that was mapped to that cluster in the training or test split. The red (dashed) bars shows data distribution for the training dataset (first few days of data) while green (dotted) bars show distribution across clusters for the later remaining days. We see a large shift in the distribution, with clusters that had only a few data points initially (e.g. cluster 5), now displaying larger participation. Similarly, other clusters which had many data points initially (e.g. cluster 2) now display a smaller percentage of the data. We note that such shift in the distribution in the embedded space is a sufficient condition for a distribution shift in the original space, and can impact model performance [31].

B. Validation in the wild

We next explore the benefits of Beagle in auto-tuning rules in practice in real-world settings with a case study based on **Dataset 1**. Here, Beagle is primed with rules from a relatively well-tuned production root causing system. Further,

in the learning process, Beagle is provided ground truth using *retrospective* analysis which is only available for 1.13% of the data points as mentioned earlier. As before, the data is collected at one minute granularities. We use the first 60% of the minutes for training, the next 20% for validation set and the final 20% for the test set.

Initial filtering rules and accuracy. In our analysis below, we focus on a subgraph containing a subset of **AGG-KEYs** where the retrospectives indicated most mispredictions were made, as they present the best opportunity for Beagle to improve the existing filters and achieve correct predictions as root causes. Figure 8a shows the relevant filters pertaining to the **AGG-KEYs** of interest: DeviceFamily (DF)-Origin, DeviceFamily (DF) and Player. Each node is annotated along with the thresholds and operators for the rule-based filters associated with that node. For instance, for a particular value of the **AGG-KEY** DF (e.g.: SmartTV) to be a root cause, the following initial conditions need to be met: $AND(X1 > 1, X2 \geq 50, X3 \geq 45, X4 < 80)$. Here, $X1, X2, X3$ and $X4$ are the relevant features of interest (and the features themselves depend on the **AGG-KEY**) such as % of anomalous traffic, # of children, etc. The graph in Figure 8a depicts the relationship across **AGG-KEYs**. For instance, since there is an edge from the **AGG-KEY** DF to the **AGG-KEY** Player, issues with a particular value of DF (e.g.: SmartTV) can be suppressed by a particular value of Player (e.g.: BrowserPlayer) if there exists an association between SmartTV and BrowserPlayer (an association exists if there are client sessions that use an BrowserPlayer and a SmartTV device family).

Beagle’s auto-tuning ability. Next, we discuss Beagle’s ability to auto-tune. We train Beagle in two steps. In the first step, we freeze the initial expert designed graph and allow Beagle to learn the thresholds for rule-based filters based on the training step. In the second step, we freeze the updated rules and allow Beagle to adapt the graph structure. Note that the data points which did not have a corresponding retrospective are marked as NA, and the loss function of Beagle treat these labels specially as discussed in §IV-E.

Figure 8b shows the changes to rules and the graph with Beagle after these steps. We make two observations. First, there are significant changes in the thresholds of the rule based filters as seen by the node annotations. Second, the weight of an edge from DF to Player is reduced from 1 to 0.14 indicating that the DF root causes should not be suppressed by the Player root causes. Note that the graph relationships used in practice are binary (0 or 1) indicating whether or not an edge is used. Beagle’s ML models however model the same in a continuous fashion, with lower values indicating the edge should not exist.

Figure 8c compares the Mistake Rate achieved by Beagle relative to the original Expert. We evaluate the Mistake Rate against the ground truth from *retrospective* analysis of anomalies. The rightmost group of bars corresponds to overall Mistake Rate, while the other three groups show the Mistake Rate per root cause category. The results show that Beagle significantly reduces the overall Mistake Rate from 0.43% to

0.05%, nearly an order of magnitude reduction. Since there are a large amount of datapoints, note that even a reduction by 0.1% is significant for a production video service. Further, the reductions are significant for each of the three categories of root causes, with mistakes completely eliminated for DF-Origin **AGG-KEY**. Finally, though we use a subset of **AGG-KEYs** in our analysis here, we verify the performance of Beagle is not at the cost of regression in other **AGG-KEYs**.

We also performed an ablation where we consider a version of Beagle, which auto-tunes rules alone rather than both the rule and graph. We find that Beagle-RuleOnly achieves a Mistake Rate of 0.12%, which is significantly lower than Expert (0.43%) but not as performant as Beagle (0.05%). The benefits with Beagle mainly come from reducing Mistake Rate for the DF root cause category – Beagle achieves a $3\times$ reduction in Mistake Rate for this category relative to Beagle-RuleOnly. This indicates the importance of tuning both the thresholds and the graph relationships.

In our discussions with the experts, the feedback about Beagle was positive. Auto-tuning abilities of Beagle were seen as greatly valuable in improving long term maintainability of the system, as Beagle reduced (i) the manual effort for tuning filters and (ii) made it easier to introduce new **AGG-KEYs** in the data. Further, experts found value in the interpretability of Beagle which allowed them to understand the suggested changes before integration in the production platform. Finally, Beagle’s usage of weighted edges in precedence graph imparted flexibility as domain experts didn’t need to manually insert or remove edges but instead could leave it to Beagle to tune.

VI. RELATED WORK

Video diagnosis and localization for multi-dimensional data. Past work has explored localization of problems related to video streaming [5] and page views [9] in settings where sessions are associated with multiple dimensions. As discussed in §II-B, these schemes use rules with statically chosen thresholds to filter anomalies, and focus on parent-child relationships determined by simple rules of thumb (where dimensions of a parent are a subset of dimensions of a child). Beagle is complementary, and focuses on automatically learning and tuning rule thresholds, and more general parent-child relationships from retrospective data.

Diagnosis in other contexts. Fault localisation in enterprise networks is well studied [33]–[41]. Sherlock [33] leverages communication patterns in a microservice architecture to create an inference graph which models the dependencies across multiple services involved in resolving a single request (e.g., a web page request). Sherlock [33] also presents an algorithm to determine the root cause based on the graph and data observations. Beagle differs in several ways. First, Beagle’s goal is to diagnose a root cause that impacts a group of users, as opposed to a single request. Second, the graph it uses is logical, that is, it captures the relationship that an issue with a particular dimension should suppress issues with other associated dimensions. Furthermore, the graph itself is based on domain knowledge. Third, a key aspect of our work is

learning from (highly limited) retrospective data, an aspect not considered in Sherlock. Our goal is to design an automated way to refine the filters and graphs using retrospective information.

VII. CONCLUSION

In this paper, our primary contribution is Beagle, a novel system for root-causing performance anomalies in the context of live video streaming. As part of Beagle, we have developed custom ML models which are interpretable and aligned with mechanisms used by domain experts. Beagle supports automatic learning and adaptation of rule thresholds and graph relationships from limited ground truth information leveraging pre-training and few-shot learning. Evaluations with large scale datasets show that (i) Beagle achieves F1 scores higher than 0.96 even when primed with fairly weak Expert models, as compared to 0.35 for GBDT, and 0.52 for Set Transformer; (ii) Beagle is effective in auto-tuning a production expert designed system reducing Mistake Rate by nearly an order of magnitude.

VIII. ACKNOWLEDGEMENT

We thank reviewers for their feedback which greatly improved the paper. This work was funded in part by Amazon and National Science Foundation (NSF) Award CNS-2212160.

REFERENCES

- [1] S. S. Krishnan and R. K. Sitaraman, “Video stream quality impacts viewer behavior: inferring causality using quasi-experimental designs,” in *Proceedings of the 2012 Internet Measurement Conference*, ser. IMC ’12. New York, NY, USA: Association for Computing Machinery, 2012, p. 211–224. [Online]. Available: <https://doi.org/10.1145/2398776.2398799>
- [2] F. Dobrian, A. Awan, D. Joseph, A. Ganjam, J. Zhan, V. Sekar, I. Stoica, and H. Zhang, “Understanding the impact of video quality on user engagement,” *Commun. ACM*, vol. 56, no. 3, p. 91–99, mar 2013. [Online]. Available: <https://doi.org/10.1145/2428556.2428577>
- [3] A. Balachandran, V. Sekar, A. Akella, S. Seshan, I. Stoica, and H. Zhang, “A quest for an internet video quality-of-experience metric,” in *Proceedings of the 11th ACM Workshop on Hot Topics in Networks*, ser. HotNets-XI. New York, NY, USA: Association for Computing Machinery, 2012, p. 97–102. [Online]. Available: <https://doi.org/10.1145/2390231.2390248>
- [4] M. Ghobadi, Y. Cheng, A. Jain, and M. Mathis, “Trickle: Rate limiting youtube video streaming,” in *Proceedings of the USENIX Annual Technical Conference*, 2012, p. 6.
- [5] J. Jiang, V. Sekar, I. Stoica, and H. Zhang, “Shedding light on the structure of internet video quality problems in the wild,” in *Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies*, ser. CoNEXT ’13. New York, NY, USA: Association for Computing Machinery, 2013, p. 357–368. [Online]. Available: <https://doi.org/10.1145/2535372.2535394>
- [6] A. A. Mahimkar, Z. Ge, A. Shaikh, J. Wang, J. Yates, Y. Zhang, and Q. Zhao, “Towards automated performance diagnosis in a large iptv network,” in *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, ser. SIGCOMM ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 231–242. [Online]. Available: <https://doi.org/10.1145/1592568.1592596>
- [7] G. Dimopoulos, I. Leontiadis, P. Barlet-Ros, K. Papagiannaki, and P. Steenkiste, “Identifying the root cause of video streaming issues on mobile devices,” in *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, ser. CoNEXT ’15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: <https://doi.org/10.1145/2716281.2836109>
- [8] J. Jiang, V. Sekar, I. Stoica, and H. Zhang, “Shedding light on the structure of internet video quality problems in the wild,” in *Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies*, ser. CoNEXT ’13. New York, NY, USA: Association for Computing Machinery, 2013, p. 357–368. [Online]. Available: <https://doi.org/10.1145/2535372.2535394>

- [9] Y. Sun, Y. Zhao, Y. Su, D. Liu, X. Nie, Y. Meng, S. Cheng, D. Pei, S. Zhang, X. Qu, and X. Guo, "Hotspot: Anomaly localization for additive kpis with multi-dimensional attributes," *IEEE Access*, vol. 6, pp. 10909–10923, 2018.
- [10] R. Bhagwan, R. Kumar, R. Ramjee, G. Varghese, S. Mohapatra, H. Manoharan, and P. Shah, "Adtributor: Revenue debugging in advertising systems," in *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. Seattle, WA: USENIX Association, Apr. 2014, pp. 43–55. [Online]. Available: <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/bhagwan>
- [11] Z. Li, J. Chen, Y. Chen, C. Luo, Y. Zhao, Y. Sun, K. Sui, X. Wang, D. Liu, X. Jin, Q. Wang, and D. Pei, "Generic and robust root cause localization for multi-dimensional data in online service systems," *Journal of Systems and Software*, vol. 203, p. 111748, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121223001437>
- [12] S. Yan, C. Shan, W. Yang, B. Xu, D. Li, L. Qiu, J. Tong, and Q. Zhang, "Cmmd: Cross-metric multi-dimensional root cause analysis," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 4310–4320. [Online]. Available: <https://doi.org/10.1145/3534678.3539109>
- [13] C. Zhang, Z. Dong, X. Peng, B. Zhang, and M. Chen, "Trace-based multi-dimensional root cause localization of performance issues in microservice systems," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639088>
- [14] J. Sun, T. Lin, Y. Liu, X. Wang, B. Jiang, L. Geng, P. Jing, and L. Dai, "iswift: Fast and accurate impact identification for large-scale cdns," in *2022 IEEE/ACM 30th International Symposium on Quality of Service (IWQoS)*, June 2022, pp. 1–10.
- [15] Y. Meng, Q. Zhang, X. Tang, W. Zhang, and J. Wang, "Ftm-rca: A fast two-stage multi-dimensional root-cause analysis of network anomalies," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, June 2023, pp. 01–10.
- [16] G. Rong, H. Wang, S. Gu, Y. Xu, J. Sun, D. Shao, and H. Zhang, "Locating anomaly clues for atypical anomalous services: An industrial exploration," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 2746–2761, July 2023.
- [17] Y. Sun, D. Cheng, P. Jin, Q. Ding, S. Zhang, X. Chen, Y. Zhang, M. Liang, D. Pei, J. Zheng, S. Luo, and X. Tang, "Robust anomaly clue localization of multi-dimensional derived measure for online video services," *IEEE Transactions on Services Computing*, vol. 16, no. 2, March 2023.
- [18] C. Liu, Y. Liu, Z. Xu, and L. Dai, "Rapminer: A generic anomaly localization mechanism for cdn system with multi-dimensional kpis," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, June 2022, pp. 319–330.
- [19] Z. Li, C. Luo, Y. Zhao, Y. Sun, K. Sui, X. Wang, D. Liu, X. Jin, Q. Wang, and D. Pei, "Generic and robust localization of multi-dimensional root causes," in *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*, Oct 2019, pp. 47–57.
- [20] X. Zheng, B. Aragam, P. K. Ravikumar, and E. P. Xing, "Dags with no tears: Continuous optimization for structure learning," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/e347c51419ffb23ca3fd5050202f9c3d-Paper.pdf
- [21] O. A. L. Analysis/IBM, "Composite conditions," <https://www.ibm.com/docs/en/oala/1.3.1?topic=utility-composite-conditions>, 2021.
- [22] I. Misra and L. v. d. Maaten, "Self-supervised learning of pretext-invariant representations," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 6707–6717.
- [23] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," in *International conference on learning representations*, 2016.
- [24] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," *Advances in neural information processing systems*, 2017.
- [25] A. Nemirovsky, "Optimization ii. numerical methods for nonlinear continuous optimization," 1999.
- [26] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf
- [27] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 3744–3753.
- [28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference for Learning Representations*, 2015.
- [29] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=Skq89Scxx>
- [30] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. ACM, Aug. 2016. [Online]. Available: <http://dx.doi.org/10.1145/2939672.2939785>
- [31] A. D'Amour, K. Heller, D. Moldovan, B. Adlam, B. Alipanahi, A. Beutel, C. Chen, J. Deaton, J. Eisenstein, M. D. Hoffman *et al.*, "Underspecification presents challenges for credibility in modern machine learning," *The Journal of Machine Learning Research*, vol. 23, no. 1, pp. 10237–10297, 2022.
- [32] M. Baktashmotlagh, M. Harandi, and M. Salzmann, "Distribution-matching embedding for visual domain adaptation," *Journal of Machine Learning Research*, vol. 17, no. 108, pp. 1–30, 2016. [Online]. Available: <http://jmlr.org/papers/v17/15-207.html>
- [33] P. Bahl, R. Chandra, A. Greenberg, S. Kandula, D. A. Maltz, and M. Zhang, "Towards highly reliable enterprise network services via inference of multi-level dependencies," in *Proceedings of the 2007 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, ser. SIGCOMM '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 13–24. [Online]. Available: <https://doi.org/10.1145/1282380.1282383>
- [34] B. Arzani, S. Ciraci, L. Chamon, Y. Zhu, H. H. Liu, J. Padhye, B. T. Loo, and G. Outhred, "007: Democratically finding the cause of packet drops," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. Renton, WA: USENIX Association, Apr. 2018, pp. 419–435. [Online]. Available: <https://www.usenix.org/conference/nsdi18/presentation/arzani>
- [35] V. Harsh, W. Zhou, S. Ashok, R. N. Mysore, B. Godfrey, and S. Banerjee, "Murphy: Performance diagnosis of distributed cloud applications," in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 438–451. [Online]. Available: <https://doi.org/10.1145/3603269.3604877>
- [36] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: practical and scalable ml-driven performance debugging in microservices," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021.
- [37] J. Gao, N. Yaseen, R. MacDavid, F. Vieira Frujeri, V. Liu, R. Bianchini, R. Aditya, X. Wang, H. L. , D. Maltz, M. Y. , and B. Arzani, "Scouts: Improving the diagnosis process through domain-customized incident routing," in *SIGCOMM*. ACM, August 2020. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/scouts-improving-the-diagnosis-process-through-domain-customized-incident-routing/>
- [38] S. Kandula, R. Mahajan, P. Verkaik, S. Agarwal, J. Padhye, and P. Bahl, "Detailed diagnosis in enterprise networks," in *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, ser. SIGCOMM '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 243–254. [Online]. Available: <https://doi.org/10.1145/1592568.1592597>
- [39] "NetNorad," <https://engineering.fb.com/2016/02/18/core-infra/netnorad-troubleshooting-networks-via-end-to-end-probing/>.
- [40] C. Tan, Z. Jin, C. Guo, T. Zhang, H. Wu, K. Deng, D. Bi, and D. Xiang, "NetBouncer: Active device and link failure localization in data center networks," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 599–614. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/tan>
- [41] H. Herodotou, B. Ding, S. Balakrishnan, G. Outhred, and P. Fitter, "Scalable near real-time failure localization of data center networks," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 1689–1698. [Online]. Available: <https://doi.org/10.1145/2623330.2623365>